

# Technical Report

---

## 1. Executive Summary

This project presents two data visualization systems developed for a university Data Visualization final project. Track A is a social network visualization based on a multi-repository GitHub developer network. Track B is a real-time cryptocurrency monitoring dashboard based on live CoinGecko market data.

Track A collects contributor data from selected open-source GitHub repositories, transforms the data into a developer-developer network, calculates network metrics, detects communities, and exports both interactive and static network visualizations. Track B polls a live REST API every 10 seconds, stores recent cryptocurrency prices, displays current metrics, and provides a threshold-based Bitcoin alert.

The project demonstrates both exploratory network visualization and operational dashboard visualization. The results should be interpreted as visual summaries of the collected data, not as complete measures of developer influence or financial market behavior.

## 2. Project Scope

The project scope includes data collection, preprocessing, visual encoding, interaction design, and output generation for two separate visualization tracks.

Track A focuses on network structure. It answers questions such as which developers appear in multiple repository contexts, which developers occupy central positions, and whether community structure emerges from shared repository contributions.

Track B focuses on real-time monitoring. It answers questions such as what the current prices of selected cryptocurrencies are, how prices have changed over the recent sliding window, and whether Bitcoin has crossed a defined short-term drop threshold.

The project does not attempt to perform predictive modeling, financial forecasting, or complete GitHub ecosystem analysis.

## 3. Visualization Design Document

The design approach follows the principle that visual encodings should match the analytical task. Track A uses a node-link diagram because the primary data structure is relational. Developers are represented as nodes, shared repository relationships are represented as edges, and network metrics are encoded visually to make structure easier to inspect.

Track B uses dashboard components because the task is monitoring current and recent values over time. Metric cards support quick reading of current prices, while the rolling line chart supports temporal comparison. A warning banner is reserved for the threshold-based alert, so the strongest visual signal appears only when attention is required.

Both tracks avoid excessive labels and decorative elements. This reduces visual clutter and supports faster interpretation.

## 4. User Persona

The primary user is a data visualization instructor, evaluator, or student reviewer who wants to inspect whether the project correctly applies data visualization methods to real data.

A secondary user is a technically literate analyst who wants to understand network structure in open-source GitHub repositories or monitor recent cryptocurrency price movement.

The user is expected to value clarity, reproducibility, and methodological transparency over visual complexity.

## 5. Visual Encoding Mappings

Track A visual encodings:

- Node: GitHub developer.
- Edge: shared contribution to at least one same repository.
- Node color: Louvain community assignment.
- Node size: degree centrality.
- Edge width: number of shared repositories.
- Label: only the top five developers by degree centrality.
- Tooltip: developer name, community, degree, degree centrality, betweenness centrality, and repositories.

Track B visual encodings:

- Metric cards: current prices for Bitcoin, Ethereum, and Solana.
- Line position: cryptocurrency price in USD over time.
- Line color: cryptocurrency symbol.
- Warning banner: Bitcoin price drop greater than 2% over approximately five minutes.
- Status indicator: API connection state.
- Timestamp: most recent update time.

Community color, node size, and edge width reduce cognitive load because they assign different visual channels to different analytical questions. Color separates groups, size highlights central actors, and width shows stronger shared-repository relationships without requiring the user to read every edge attribute.

## 6. Social Network Visualization Overview

Track A constructs a social network from GitHub contributor data. The selected repositories are:

- `pallets/flask`
- `django/django`
- `fastapi/fastapi`
- `pandas-dev/pandas`
- `scikit-learn/scikit-learn`

The network is based on developer co-contribution patterns across these repositories. If two developers contributed to at least one same repository, they are connected in the projected developer-developer network.

The final visualization is exported in two forms: an interactive HTML file for exploration and a static PNG file for report inclusion.

## 7. Dataset and Data Provenance

The dataset is collected from the GitHub REST API using the contributors endpoint for each selected repository. The project uses real public GitHub data, not a built-in toy network such as Zachary Karate Club.

For each repository, the script collects up to 40 contributors to reduce network density and keep the visualization readable. Each raw record includes:

- developer username
- repository name
- contribution count
- GitHub profile URL

The raw output is saved as:

```
track_a_social_network/data/raw_contributors.csv
```

Because GitHub API responses can change over time, final numerical results may vary depending on the date of collection and API availability.

## 8. Edge List and Node Table Explanation

The raw data is first treated as a bipartite structure:

```
Developer -> Repository
```

This is then projected into a developer-developer network:

```
Developer -> Developer
```

An edge exists when two developers contributed to at least one same repository. The edge weight is the number of repositories shared by the two developers.

The edge list is saved as:

```
track_a_social_network/data/edge_list.csv
```

It contains:

- `source`
- `target`

- `weight`
- `shared_repositories`

The node table is saved as:

```
track_a_social_network/data/node_table.csv
```

It contains developer-level attributes, including repositories, repository count, total contributions, degree, degree centrality, betweenness centrality, and community assignment.

## 9. Community Detection Method

The project applies Louvain community detection using edge weights. Louvain is appropriate for this network because it identifies groups of nodes that are more densely connected to each other than to the rest of the graph.

In this project, communities can be interpreted as clusters of developers who share similar repository contribution contexts. The detected communities should not be treated as formal teams or organizations unless supported by additional evidence.

Community color is used in the visualization to help users distinguish clusters quickly.

## 10. Centrality Method

The project calculates two centrality measures:

- Degree centrality
- Betweenness centrality

Degree centrality measures how many direct connections a developer has relative to the size of the network. It is used for node size because it highlights developers who are broadly connected through shared repositories.

Betweenness centrality measures how often a developer lies on shortest paths between other developers. It is useful for identifying possible bridge actors who connect different parts of the network.

Final values should be copied from:

```
track_a_social_network/data/node_table.csv
```

Example placeholder:

```
Top degree centrality developer: [insert top developer from node_table.csv]  
Top betweenness centrality developer: [insert top bridge developer from  
node_table.csv]
```

## 11. Layout Justification

The visualization uses a force-directed layout. Force-directed layouts position connected nodes closer together and push less-connected nodes farther apart. This is useful for social network visualization because it can reveal clusters, dense regions, and bridge nodes.

In this project, the layout supports the interpretation of community separation. Developers with many shared repository relationships tend to appear near related developers, while bridge actors may appear between clusters. This makes the structure easier to inspect than a random or alphabetical layout.

The layout should be interpreted as an aid to visual exploration, not as a precise spatial measurement.

## 12. Community Narrative

The Louvain communities represent groups of developers connected through shared repository contribution patterns. A community may correspond to developers concentrated around one major repository or to developers who appear across multiple repositories.

After running the script, the final community narrative should be completed using the generated node table and visualization. Example placeholders:

- Largest community: [insert community ID and description]
- Most visually central community: [insert community ID]
- Community containing the top degree centrality developer: [insert community ID]

The narrative should focus on observable structure rather than unsupported assumptions about developer roles.

## 13. Key Actors

Key actors are developers with high centrality values. A developer with high degree centrality is directly connected to many others. A developer with high betweenness centrality may act as a bridge between groups.

The final report should identify key actors from the generated node table:

- Top developer by degree centrality: [insert top developer from node\_table.csv]
- Top developer by betweenness centrality: [insert top developer from node\_table.csv]
- Developer appearing in the most repositories: [insert developer from node\_table.csv]

These actors should be described carefully as central within this collected dataset, not necessarily central across all of GitHub.

## 14. Weak Link / Bridge Analysis

Bridge analysis focuses on developers or edges that connect otherwise separate communities. Betweenness centrality is the main measure used for this purpose.

A high-betweenness developer may indicate cross-project participation or a structural position between clusters. However, the project does not infer motivation or organizational role from the network alone.

Placeholder for final interpretation:

```
Primary bridge actor: [insert high betweenness developer]
Possible bridge communities: [insert connected community IDs]
Relevant shared repositories: [insert repositories from edge_list.csv]
```

Weak links can also be examined through low-weight edges. A weight of 1 indicates that two developers share only one repository, while higher edge weights indicate broader overlap.

## 15. Visualization Critique

The Track A visualization is effective for showing community structure, central actors, and shared contribution relationships. The use of node color for community, node size for degree centrality, and edge width for shared repository count supports multiple levels of interpretation.

The main limitation is potential density. Even after limiting contributors per repository, projected developer networks can become visually crowded because a single repository with many contributors creates many pairwise edges. The project reduces clutter by limiting labels to the top five developers by degree centrality and by using tooltips for detailed information.

The static PNG is useful for documentation, while the interactive HTML is better for exploration.

## 16. Track A Improvements

Potential improvements include:

- Adding filters for repository, community, or minimum edge weight.
- Allowing users to adjust the contributor limit.
- Adding a legend explaining community colors and node size.
- Comparing results across different collection dates.
- Including additional GitHub metadata such as commit dates or contributor types.
- Validating whether high centrality corresponds to meaningful project roles.

## 17. Real-Time Dashboard Overview

Track B is a real-time dashboard for monitoring selected cryptocurrency prices. It tracks Bitcoin, Ethereum, and Solana in USD using CoinGecko live data.

The dashboard includes:

- Current price metrics
- API connection status
- Last updated timestamp
- Rolling line chart
- Bitcoin threshold alert

The purpose is to demonstrate real-time data visualization, not to provide financial advice.

## 18. API Source and Data Pipeline

Track B uses the free CoinGecko simple price API. The dashboard requests current USD prices for:

- bitcoin
- ethereum
- solana

The data pipeline is:

```
CoinGecko API -> requests -> pandas DataFrame -> CSV persistence -> Streamlit UI -> Plotly chart
```

Recent observations are stored in:

```
track_b_realtime_dashboard/data/live_crypto_data.csv
```

This CSV persistence allows the dashboard to maintain recent history across refreshes.

## 19. REST Polling and Refresh Rate Justification

The dashboard uses REST polling every 10 seconds through `streamlit-autorefresh`. This interval is frequent enough to create a real-time monitoring experience while avoiding excessive API requests.

A shorter interval could increase responsiveness but may create unnecessary API load and possible rate-limit problems. A longer interval would reduce API pressure but make the dashboard less useful for short-term monitoring.

The 10-second interval is therefore a practical compromise for a university dashboard project.

## 20. Sliding Window Explanation

The dashboard keeps a sliding window of recent data points for each tracked cryptocurrency. This prevents the chart and CSV file from growing indefinitely.

The sliding window also supports readability. Instead of showing all historical observations, the dashboard focuses on recent movement, which is more appropriate for real-time monitoring.

The current implementation keeps up to 90 recent observations per coin. At a 10-second polling rate, this provides enough history for the approximate five-minute Bitcoin alert calculation while keeping the interface compact.

## 21. Alert Threshold Logic

The alert logic checks whether Bitcoin has dropped more than 2% compared to approximately five minutes earlier.

The percentage change is calculated as:

```
((current BTC price - previous BTC price) / previous BTC price) * 100
```

If the result is less than or equal to  $-2.0$ , the dashboard displays a red warning banner. This strong visual treatment is reserved for alert conditions only. Under normal conditions, the dashboard uses muted colors and text to avoid unnecessary alarm.

The threshold is simple and interpretable. It is not intended as a trading signal.

## 22. Dashboard UX and Cognitive Load

The dashboard uses a clean layout to reduce cognitive load. Current prices are presented as metric cards because they are single-value indicators. The rolling line chart is used for temporal comparison. Connection status and last updated time are placed near the top so users can quickly judge whether the displayed data is fresh.

Muted line colors are used for normal data. A strong red warning is used only when the Bitcoin alert condition is met. This contrast helps users separate routine monitoring from urgent conditions.

## 23. Performance and Latency

The dashboard performs lightweight API requests and stores only a limited sliding window of observations. This keeps memory usage low and makes the dashboard suitable for local execution.

Latency depends on CoinGecko API response time, local network conditions, and Streamlit refresh behavior. If the API request fails, the dashboard handles the failure gracefully by showing an unavailable connection status and preserving recent stored data when available.

## 24. Future Scaling Analysis

If the dashboard were expanded, several scaling issues would need to be considered:

- More cryptocurrencies would increase chart complexity and API payload size.
- Longer historical windows would require more efficient storage.
- Multiple users would require a shared backend rather than local CSV persistence.
- More advanced alerting would require configurable thresholds and notification channels.
- API rate limits would become more important in a deployed environment.

For the current academic scope, local CSV persistence and a small set of tracked coins are sufficient.

## 25. Track B Limitations and Improvements

Track B limitations include:

- It depends on CoinGecko API availability.
- It stores data locally rather than in a database.
- It tracks only three cryptocurrencies.
- The alert threshold is fixed rather than user-configurable.
- The dashboard does not include long-term historical analysis.

Potential improvements include:

- Adding configurable coin selection.
- Adding user-controlled alert thresholds.



- Adding percent-change metrics for each coin.
- Using a database for longer-running deployments.
- Adding export options for dashboard data.

## 26. Conclusion

This project demonstrates two important forms of data visualization. Track A shows how relational data can be transformed into a social network, analyzed with centrality and community detection, and visualized with meaningful encodings. Track B shows how live API data can be monitored through a real-time dashboard with current metrics, a rolling chart, and threshold-based alerting.

Together, the two tracks show the value of matching visualization design to data type and user task. Network data benefits from community color, node size, edge width, and force-directed layout. Real-time monitoring data benefits from metric cards, time-series charts, status indicators, and focused alerts.

Final interpretation should be completed after running the scripts and copying the generated values from the output CSV files.